Goal:

- Implement Swagger & GraphQL

Scenario:

- Setup Swagger for a single REST endpoint, include proper annotations so Swagger can include them in documentation
- Setup GraphQL and create a single query + mutation
- Recommended to do this on a copy of our starter projects

Dependencies for Swagger API in Maven

```
        <dependency>
            <groupId>org.springdoc</groupId>
            <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
            <version>2.8.13</version>
        </dependency>
```

Self Note: "mvn clean install" when adding new dependencies

Run the docker and make sure the database is running then launch the app with "mvn spring-boot:run"

Finally check the end point:

http://localhost:8080/swagger-ui/index.html

Swagger Imports for the controllers

```
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
```

Self Note: don't mix up Response and Responses pay attention to the S..

I added this above Get mapping /{id}

```
@Operation(
    summary = "Get a student by their ID",
    description = "Retrieves a student using their unique ID"
)
@ApiResponses({
    @ApiResponse(
        responseCode = "200",
        description = "Student retrieved successfully"
    ),
    @ApiResponse(
        responseCode = "400",
        description = "Student not found"
```

```
    )
})
```

Dependencies for Swagger API in Maven

```
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-graphql</artifactId>
        </dependency>
```

Add this to the application.properties

```
spring.graphql.path=/api/graphql
```

I can test it here http://localhost:8080/graphql

Here's a schema.graphqls

```
type Query {
    student(id: ID!): Student
}

type Mutation {
    createStudent(
        firstName: String!
        lastName: String!
        email: String!
        schoolId: Int
    ): Student
}

type Student {
    id: ID!
    firstName: String
    lastName: String
    email: String
}
```

Notes:

`Query` defines data retrieval operations.

`Mutation` defines operations that change data.

`ID!` means the ID is required.

`String!` means the string argument is required.
`Student` describes the fields the client can request.

Using thunder client we can interact with GraphQL once it's setup

